The Journal of Systems and Software 231 (2026) 112646

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

"Digital Camouflage": The LLVM challenge in LLM-based malware detection[☆,☆☆]

Ekin Böke[a,c,*], Simon Torka[b]

[a] Applied Machine Learning Department, Fraunhofer Heinrich-Hertz-Institute (HHI), 10587 Berlin, Germany
[b] Distributed Artificial Intelligence Laboratory/Technische Universität Berlin, Germany
[c] Technische Universität Berlin, Germany

ARTICLE INFO

Keywords:
Malware detection
Large Language Models
Cybersecurity
LLVM

ABSTRACT

Large Language Models (LLMs) have emerged as promising tools for malware detection by analyzing code semantics, identifying vulnerabilities, and adapting to evolving threats. However, their reliability under adversarial compiler-level obfuscation is yet to be discovered. In this study, we empirically evaluate the robustness of three state-of-the-art LLMs: ChatGPT-4o, Gemini Flash 2.5, and Claude Sonnet 4 against compiler-level obfuscation techniques implemented via the LLVM infrastructure. These include control flow flattening, bogus control flow injection, instruction substitution, and split basic blocks, which are widely used to evade detection while preserving malicious behavior. We perform a structured evaluation on 40 C functions (20 vulnerable, 20 secure) sourced from the Devign dataset and obfuscated using LLVM passes. Our results show that these models often fail to correctly classify obfuscated code, with precision, recall, and F1-score dropping significantly after transformation. This reveals a critical limitation: LLMs, despite their language understanding capabilities, can be easily misled by compiler-based obfuscation strategies. To promote reproducibility, we release all evaluation scripts, prompts, and obfuscated code samples in a public repository. We also discuss the implications of these findings for adversarial threat modeling, and outline future directions such as software watermarking, compiler-aware defenses, and obfuscation-resilient model design.

1. Introduction

Malware detection and mitigation are important parts of modern cybersecurity efforts. During the past decade, LLMs have emerged as powerful tools in this field. These models excel at identifying deviations from the norm, referred to as anomalies. Recent research even shows the possibility of using LLMs as code executers and analyzing code and detecting anomalies by executing code and comparing expected behavior and actual output (Lyu et al., 2024).

Unlike traditional signature-based detection, LLMs leverage their training on diverse datasets to identify threats, even in previously unseen contexts. This makes them particularly useful in dynamic and rapidly evolving threat landscapes. Yet, their effectiveness crucially depends on the structure and transparency of the code they analyze. A key factor is the role of compiler-generated intermediate representations, which form the level where both optimizations and obfuscation techniques are applied.

LLVM, a widely used compiler framework, first translates C source code (`.c`) into an intermediate representation known as LLVM Intermediate Representation (LLVM IR) (LLVM Project, 2025). The LLVM IR exists in two equivalent forms: as a human-readable, assembler-like format (`.ll`) and as a compact, binary-encoded variant (`.bc`). Optimization and obfuscation passes act directly at this level before the program is translated into platform-specific binary files such as object files (`.o`) or executable files (`.exe`). LLVM IR encodes program semantics at a low but still abstract level, making it suitable not only for standard compiler transformations but also for obfuscation techniques such as control flow smoothing, command substitution, or the insertion of dead code, which can significantly obscure syntactic and semantic features. This makes it difficult for LLM-based detection models to reliably identify malicious code in obfuscated LLVM code, which is a key weakness of current approaches.

This growing use of sophisticated obfuscation calls for a deeper examination of how current malware detection techniques especially those based on LLMs cope with adversarial transformations.

☆ This article is part of a Special issue entitled: 'Reliable and Secure LLMs for SE' published in The Journal of Systems & Software.
☆☆ Editor: Raffaela Mirandola.

* Corresponding author.
E-mail addresses: ekin.boeke@hhi.fraunhofer.de (E. Böke), torka@tu-berlin.de (S. Torka).

https://doi.org/10.1016/j.jss.2025.112646
Received 29 March 2025; Received in revised form 15 September 2025; Accepted 19 September 2025
Available online 27 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

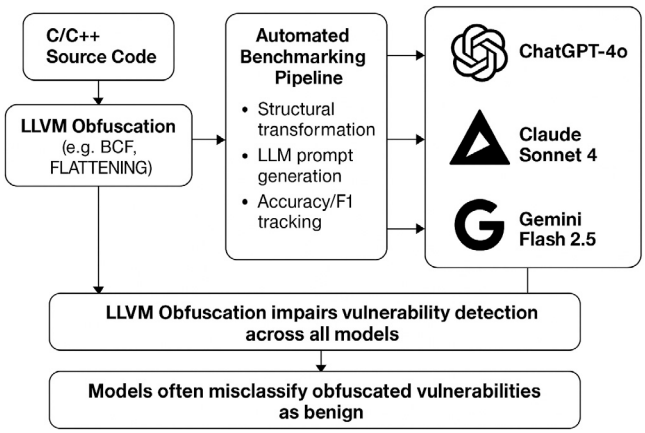

Fig. 1. Overview of our benchmarking pipeline.
Logos are trademarks of their respective owners. Used here for illustrative purposes only.

To establish this context, Section 2 provides a background on malware detection techniques and their evolution over time, highlighting how advances in natural language processing (NLP) enabled the emergence of LLM-based detection methods.

Section 3 introduces malware obfuscation techniques and evasion strategies commonly used to bypass detection systems. A key focus is placed on the role of LLVM in code transformation, particularly how its optimization and obfuscation capabilities can be leveraged to alter malicious code while maintaining functionality.

Building on this foundation, Section 4 defines the adversarial threat model underpinning our study. We consider a Man-At-The-End (MATE) attacker. In this scenario, the defender controls the analysis environment: both the compiled binaries and their corresponding LLVM Intermediate Representation (LLVM IR). The attacker, on the other hand, has direct access to the software and deliberately employs compiler-level obfuscation techniques to evade both static and dynamic scrutiny.

This scenario reflects realistic operational conditions. In real-world investigations, security analysts often encounter obfuscated binaries obtained from malware repositories, phishing campaigns, or compromised systems. However, these raw binary files cannot be directly analyzed using LLMs, which are specialized for natural language processing and cannot interpret binary code without specialized training. In contrast, the LLVM IR format, especially its textual `.ll` representation offers a middle ground: it encodes low-level semantics in a format that is both human-readable and parsable by LLMs. While `.ll` files are typically not distributed with binary payloads, they can be recovered through leaked source code, builder toolkits, or reverse engineering. For example, Toor (2022) demonstrates that tools like National Security Agency (2019) specifically Ghidrall can reconstruct LLVM IR from binaries using Ghidra's P-Code and decompilation trees, enabling downstream analysis even in the absence of original source code.

To systematically evaluate the impact of such transformations on LLM-based vulnerability detection, we design a controlled benchmarking setup leveraging representative compiler-level obfuscation passes and real-world vulnerable code samples.

Section 5 presents a comprehensive benchmarking study that evaluates the robustness of three state-of-the-art LLMs: *ChatGPT-4o* (OpenAI, 2024), *Gemini Flash 2.5* (DeepMind, 2024), and *Claude Sonnet 4* (Anthropic, 2024) against a set of adversarially obfuscated code samples. To clarify our experimental setup, Fig. 1 illustrates the full pipeline from code selection to evaluation.

In our obfuscation pipeline, we rely on the main branch of the publicly available *Obfuscator-LLVM* project (Obfuscator-LLVM Team, 2014), compatible with LLVM 14. This version supports four stable and well-documented transformation passes: control-flow flattening, bogus control-flow insertion, instruction substitution, and basic-block splitting. These passes are widely studied and have been observed in both academic literature and real-world malware (Revay, 2023), offering a representative sample of contemporary obfuscation strategies.

To provide meaningful input to the models, we sampled 40 functions (20 labeled as vulnerable, 20 as safe) from the Devign dataset (Zhou et al., 2019). Minimal modifications were made to ensure compilability, such as inserting missing type declarations, without changing the original program logic or semantics. A detailed rationale for this selection, along with implementation details, is provided in Section 5.2.

Each function, both in its original and obfuscated form, was evaluated in a zero-shot setting using the three LLMs. Since LLMs are inherently generative and non-deterministic across sessions, we repeated each evaluation three times using separate chat sessions to ensure result consistency.

We evaluate model outputs using standard classification metrics: precision, recall, and F1 score before and after obfuscation. The results reveal a substantial degradation in performance for certain models, underscoring the vulnerability of LLM-based detection systems to compiler-level adversarial transformations.

Section 6 discusses the limitations of our study, including dataset size, pass diversity, and the use of zero-shot prompting. These considerations contextualize the scope and generalizability of our findings.

Finally, Section 7 summarizes our conclusions and highlights promising directions for future work. These include the use of compiler-level obfuscation for software protection, software watermarking, adversarial hardening of LLM-based security tools, and training models directly on low-level representations such as LLVM IR to improve resilience against transformation-based attacks.

2. Evolution of LLM based malware detection

Malware detection has been a fundamental aspect of cybersecurity since the widespread adoption of the Internet, growing alongside the increasing complexity of malicious software. Traditional approaches relied on simpler signature-based methods. Although effective against previously identified threats, this approach faced significant challenges with zero-day attacks, where no prior signature exists, and polymorphic malware, which alters its code structure while maintaining its functionality. A key limitation of signature-based methods is their inherently reactive nature as malware techniques became increasingly complex, the field has found the solution in machine learning (ML). ML began to address the limitations of traditional methods by learning patterns in data that can be used for classification purposes. Common techniques included static analysis, which involves analyzing code structure and features without executing the program; dynamic analysis, which observes runtime behavior to identify malicious actions; and hybrid analysis, which combines both methods. Early machine learning models, such as Support Vector Machines (SVMs) and Random Forests, proved effective in classification with extracted features from code. However, these models struggled to generalize to large datasets. The rise of big data brought with it challenges, as highlighted by studies such as Anderson and Cafarella (2016), which underlined the importance of robust feature engineering. The study by Raff et al. (2017) introduced MalConv, a deep learning model designed for malware detection by analyzing raw binary files. Although not a LLM, MalConv demonstrated the effectiveness of direct input analysis, bypassing traditional manual feature extraction. This approach shares conceptual similarities with models based on NLP, which process long sequences and capture complex dependencies. Recent advances in LLMs build on these ideas, leveraging their ability to analyze entire code structures holistically, identify malicious intent, and capture global patterns. For example, the study by Singh et al. (2024) demonstrates the application of LLMs in the analysis of Java code for malicious behaviors, highlighting their capacity to handle complex code patterns. By integrating improvements in sequence modeling and contextual understanding, LLMs have opened new possibilities for malware analysis, offering enhanced capabilities to detect obfuscated or complex malicious behavior.

2.1. LLM-based malware detection

LLMs, such as GPT-3 (Brown et al., 2020) and BERT (Devlin et al., 2018), have influenced malware detection using their ability to process large-scale statistical representations of source code. Although LLMs do not inherently "understand" code as humans do, they can capture patterns and relationships within code syntax and semantics through pretraining on extensive datasets. Transformer-based models such as CodeBERT (Feng et al., 2020) and CodeGPT (Lu et al., 2021) have been fine-tuned on code corpora to support downstream tasks like vulnerability detection and automated code repair, though they are typically used in supervised or pipeline-based settings rather than through direct user interaction.

A key strength of LLMs lies in processing code as token sequences, similar to NLP tasks. Pre-training on large data sets of source code and programming documentation enables these models to capture latent patterns and understand software behaviors. LLMs lack explicit program flow modeling and semantic understanding at the level of tools such as abstract syntax trees (ASTs) or control flow graphs (CFGs). Operating primarily at the token or sequence level, they can overlook deeper structural relationships unless fine-tuned on task-specific datasets designed to emphasize these aspects.

One such domain-specific model, CodeBERT (Feng et al., 2020), exemplifies how LLMs tailored to programming languages can overcome some of these limitations. By leveraging token embeddings and attention mechanisms, CodeBERT effectively detects patterns in code, including obfuscated or incomplete samples. While it does not explicitly model program flow semantics, its capacity to capture token relationships has demonstrated significant advantages in tasks like vulnerability detection and code understanding. For example, in the work by Li et al. (2024), CodeBERT was successfully applied to detect SQL injection vulnerabilities. By embedding code representations and utilizing datasets from real-world repositories, the model achieved notable improvements in detection performance and training efficiency, outperforming traditional static application security testing (SAST) tools. This reinforces the potential of deep learning models like CodeBERT to transform security analysis and elevate software development practices.

Building on the success of CodeBERT, MalBERT (Rahali and Akhloufi, 2021) is a domain-specific adaptation designed for malware detection by leveraging its ability to analyze token relationships in code. The model is fine-tuned using the Androzoo public dataset (Allix et al., 2016) to capture patterns and signatures characteristic of malicious code in Android applications, such as API misuse, control flow obfuscation, and embedded malicious payloads. This domain-specific pre-training enables MalBERT to effectively identify malware, including detecting suspicious permissions or obfuscated strings. Its success demonstrates how domain-specific adaptations of LLMs can enhance their effectiveness in niche areas, addressing limitations seen in general-purpose models.

Similarly, in a study by Demirci et al. (2022), a domain-specific GPT-2 model (DSLM-GPT2) was pre-trained on assembly instructions to capture contextual dependencies. Although DSLM-GPT2 achieved an F1 score of 86.0% in sentence-level analysis, it was outperformed by a document-level analysis model (DLAM) based on stacked Bidirectional Long Short-Term Memory (BiLSTM) networks, which achieved an F1 score of 98.3%. This highlights the importance of holistic understanding of the data and the ability to model sequential dependencies effectively. The findings underscore that, while LLMs like GPT-2 excel at capturing token-level relationships, complementary approaches like BiLSTM, which explicitly model sequential dependencies, may better address tasks requiring a broader contextual understanding, such as static analysis of code.

LLMs show great potential in cybersecurity due to their ability to generalize across various programming languages and recognize patterns resembling novel malware. Studies such as Akinsowon and Jiang (2024) have demonstrated that LLMs can differentiate known malware from novel families, thus achieving high accuracy rates even with previously unseen malware.

Similarly, research like Li et al. (2022) highlights that models can detect adversarial malware variants across multiple subtypes, highlighting their ability to generalize and identify new threats. In the paper Adversarially Robust Multi view Malware Defense (ARMD) a new multi view learning framework that enhances the robustness of deep learning based malware detectors against adversarial variants is presented. By using the multiple views of a malware file, such as the source code view alongside the binary view, ARMD exploits the fact that adversarial modifications often leave certain characteristics in the source code unchanged. This approach significantly improves the detection of adversarial malware variants. Experiments on three renowned open source deep learning-based malware detectors across six common malware categories demonstrate that ARMD can improve adversarial robustness by up to seven times. This further solidifies the idea that the effectiveness of LLMs is highly dependent on the diversity and completeness of their training data. While they excel at recognizing familiar patterns, their ability to detect entirely new or significantly different malware without additional fine-tuning may be limited in the case of data sparsity. Challenges such as code obfuscation and the substantial computational resources required for real-time deployment further hinder their widespread use.

While such hybrid frameworks that integrate LLMs with holistic analysis techniques offer improvements in feature representation and generalization, they often fall short when facing obfuscated malware. In particular, LLVM-based code transformations can severely distort syntactic and semantic patterns that LLMs rely on. These challenges underscore the need to systematically examine how resilient current LLM-based approaches are when confronted with obfuscation.

To this end, we deliberately chose to evaluate general-purpose LLMs such as ChatGPT-4o (OpenAI, 2024), Gemini Flash 2.5 (DeepMind, 2024), and Claude Sonnet 4 (Anthropic, 2024) rather than specialized models such as CodeBERT (Feng et al., 2020) or MalBERT (Rahali and Akhloufi, 2021). While the latter have been fine-tuned for specific downstream tasks such as code search, summarization, or vulnerability detection, the generalist models we investigate represent the current state of the art in universal, dialog-oriented LLMs. These models are increasingly applied in real-world software engineering practice for tasks ranging from code review and debugging to security analysis. Consequently, their robustness against code obfuscation is of immediate relevance to practitioners and the broader research community. To the best of our knowledge, this is the first systematic study assessing how LLVM-based obfuscation impacts malware detection with general-purpose LLMs, thereby filling a gap in existing research that has so far focused primarily on unobfuscated code or on specialized, task-specific models.

3. Malware obfuscation techniques

Malware obfuscation techniques are used by attackers to evade detection and analysis. By modifying the structure, appearance, and behavior of malicious code, obfuscation can make it significantly more difficult for security analysts to recognize and eliminate threats. These techniques range from simple syntactic changes to advanced transformations that adapt according to the execution environment. Understanding them is essential for developing effective countermeasures and improving malware detection strategies. In this chapter, we will explore various malware obfuscation techniques. Detailed examples of obfuscation techniques are provided in Appendix A.

3.1. Introduction to malware obfuscation

Malware obfuscation is a set of techniques designed to conceal malicious code and evade detection. These techniques modify the structure, appearance, or behavior of malware without altering its functionality,

Table 1
Simple C code.

```
#include <stdio.h>

int calculate(int a, int b) {
  return a + b;
}

int main() {
  int result = calculate(5, 3);
  printf(‘‘Result: %d\n’’, result);
  return 0;
}
```

complicating both static and dynamic analysis. Obfuscation methods can be broadly categorized into **syntactic**, **semantic**, and **behavioral** transformations:

- **Syntactic Obfuscation**: Alters the structure of code while keeping its logic intact. This includes *variable renaming*, *dead code insertion*, and *formatting changes* to mislead static analysis tools. Although these transformations do not modify the program's behavior, they can significantly hinder readability and make reverse engineering more time-consuming.
- **Semantic Obfuscation**: Modifies how code executes by introducing *control flow distortions*, *opaque predicates*, and *instruction substitution*, making analysis significantly harder for reverse engineering and decompilation tools. These techniques not only obscure the program's logic but also increase the computational overhead required for security tools to analyze the code effectively.
- **Behavioral Obfuscation**: Affects runtime behavior through *encryption*, *packing*, and *polymorphism/metamorphism*, ensuring that the malicious payload remains hidden until the time of execution. By dynamically decrypting or unpacking code in memory, these techniques evade static analysis and signature-based detection. This approach allows malware to adapt and mutate, making it a challenge for traditional security tools to recognize and neutralize threats. Additionally, some advanced obfuscation methods incorporate environment-aware execution, ensuring that the malicious payload activates only under specific conditions, further complicating detection efforts.

These techniques force security tools to rely on more advanced heuristics, AI-driven anomaly detection, or runtime monitoring to catch malicious activity.

3.2. LLVM and its role in code transformation

LLVM provides a powerful compiler infrastructure for creating a variety of compiler tools, such as optimizers, debuggers, and static analyzers. Due to its modular architecture, LLVM has become a widely used platform for building sophisticated software tools, including those designed for obfuscating and transforming code. The flexibility of LLVM makes it particularly appealing for tasks requiring fine-grained control over code manipulation (Lattner and Adve, 2004).

LLVM operates using an Intermediate Representation (IR), a platform-independent abstraction layer that decouples source code from target machine code. This design enables cross-language compatibility and allows for advanced optimizations across multiple programming languages. LLVM IR is a low-level, strongly-typed representation based on Static Single Assignment (SSA) form, ensuring efficient transformations and analysis while serving as a bridge between high-level code and machine instructions.

To better understand how LLVM IR works, we consider a simple C program, which is shown in Table 1.

The corresponding LLVM IR representation, obtained by compiling the code with the `-emit-llvm` flag in Clang, is provided in Table 2.

Table 2
LLVM IR representation of a simple C code.

```
; Function to add two integers

define i32 @calculate(i32%a, i32%b) {
  entry:
  %sum = add i32%a, %b
  ret i32%sum
}

; Main function

define i32 @main() {
  entry:
  %result = call i32 @calculate(i32 5, i32 3)
  %formatStr = alloca [10 x i8], align 1
  store [10 x i8] c‘‘Result: %d\0A\00’’, ptr %formatStr
  call i32 (ptr, . . . ) @printf(ptr %formatStr, i32%result)
  ret i32 0
}

declare i32 @printf(ptr, . . . )
```

LLVM IR incorporates several key features that make it a powerful intermediate representation for compiler optimizations and analysis. One of its core principles is SSA form, where each variable is assigned exactly once. This ensures better optimization and analysis, as seen in the instruction %sum = add i32%a, %b, where %sum remains immutable throughout execution. LLVM IR is also strongly typed, meaning that each variable has a specific type. For example, in the function @add, the parameters %a and %b are explicitly declared as i32 (32-bit integers), enforcing strict type safety. Additionally, LLVM IR employs explicit memory management, avoiding high-level memory abstractions. Instead, it uses the alloca instruction for manual memory allocation, as demonstrated in %formatStr = alloca [10 x i8], align 1, which reserves stack space for a string. Function calls in LLVM IR are also explicitly defined; for instance, the instruction %result = call i32 @calculate(i32 5, i32 3) clearly invokes the @calculate function, providing a low-level representation of function execution. Finally, LLVM IR supports instruction-level optimizations, including function inlining, constant propagation, and dead code elimination, ensuring efficient machine code generation before execution. These features make LLVM IR a robust and flexible foundation for program transformations, security mechanisms, and advanced compiler optimizations.

The ability to manipulate this intermediate representation allows LLVM to serve as the foundation for complex code transformations, enabling malware authors to employ advanced obfuscation techniques to evade detection by static analysis tools, as in the work of Sharif et al. (2008). They introduce a technique called Conditional Code Obfuscation (CCO), which alters the execution of a program according to its environment. Malware authors can leverage Conditional Code Obfuscation (CCO) to alter a program's behavior based on its execution environment, allowing malicious code to evade detection in sandboxes or debuggers while operating normally in real-world scenarios. The authors implemented CCO using an LLVM-based framework, enabling obfuscation techniques to be applied directly at the compiler level. Their evaluation against common malware analysis tools, including IDA Pro (Hex-Rays, 2023) and static disassemblers, demonstrated that these obfuscation methods significantly increase analysis time and obstruct the reconstruction of control flow graphs, which static analysis tools rely on. The paper concludes that CCO and similar obfuscation techniques present a formidable challenge to modern malware analysis. While CCO, focuses on dynamically altering program execution, another prominent obfuscation approach at the compiler level is Obfuscator-LLVM (O-LLVM) (Junod et al., 2015). Developed as an extension to the LLVM compilation framework, O-LLVM systematically applies obfuscation transformations at the IR level, integrating them into the compilation process rather than modifying the binary afterwards.

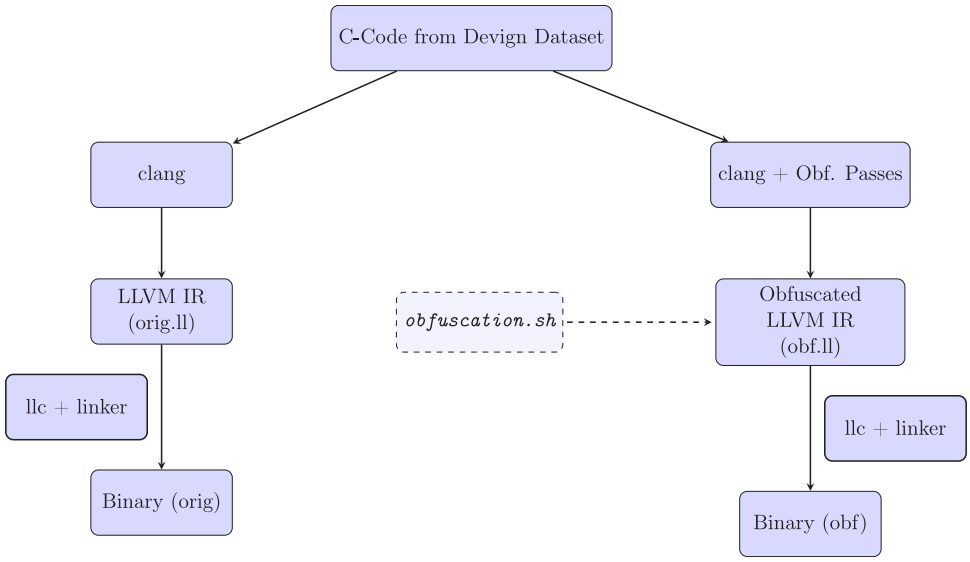

Fig. 2. Transformation pipeline from C source code to original and obfuscated LLVM IR using `obfuscation.sh`. The IR is compiled to binary using `llc` and a linker.

Unlike CCO, which relies on runtime environment checks to alter execution behavior, O-LLVM applies static code transformations that increase the difficulty of reverse engineering. By leveraging LLVM's modular compilation pipeline, O-LLVM implements obfuscation techniques designed to hinder analysis tools such as IDA Pro and National Security Agency (2019). These include:

- **Control Flow Flattening:** Restructures a program's control flow into a dispatcher-based model, making it difficult to reconstruct the original branching structure.
- **Instruction Substitution:** Replaces standard operations with functionally equivalent but more complex alternatives, disrupting pattern-based analysis.
- **Bogus Control-Flow Insertion:** Introduces opaque predicates to create misleading execution paths that confuse decompilers.
- **Procedure Merging:** Combines multiple functions into a single unit, obscuring logical program structure.
- **Tamper-Proofing Mechanisms:** Embeds runtime integrity checks to detect unauthorized modifications and potential analysis attempts.

Junod et al. (2015) evaluate the effectiveness of these transformations and highlight their ability to significantly increase reverse engineering effort while introducing only moderate runtime overhead. Their findings demonstrate that compiler-assisted obfuscation provides a structured alternative to traditional binary obfuscation techniques by ensuring that transformations are deeply integrated into the program's structure before code generation.

As LLVM-based obfuscation techniques continue to evolve, they play a crucial role in the broader field of software security by complicating static analysis and reverse engineering. The continued development of these methods highlights the ongoing arms race between obfuscation and de-obfuscation, reinforcing the need for more advanced reverse engineering tools in modern security research. The arms race is evident not only in research but also in the threat landscape, where attacker use similar techniques to complicate analysis and detection (see Section 4).

4. Adversarial threat model

This study is grounded in a realistic MATE threat model, inspired by the tactics of APTs. We assume the attacker has full control over the software build environment, including the ability to alter the source code and apply compiler-level obfuscation during compilation. Such an assumption is motivated, as prior research shows that modern malware and mutation engines routinely employ such transformations. Brezinski's metamorphic malware survey highlights that malware authors widely use techniques such as bogus control flow insertion, instruction substitution, and basic block transformations to avoid detection and complicate analysis pipelines (Brezinski, 2023). Complementing this perspective, Zhang et al. demonstrate in their Khaos framework that LLVM-based obfuscations degrade the effectiveness of binary diffing techniques, a core method for vulnerability discovery (Zhang et al., 2023).

Evidence from the field further substantiates this threat model: the *ANEL* backdoor, attributed to APT10, has been shown to employ compiler-level obfuscations including opaque predicates and control-flow flattening to resist both static and dynamic analysis (VMware Threat Analysis Unit, 2019). Taken together, these findings indicate that our assumed adversarial capabilities are not only realistic but also reflective of techniques already observed in academic studies and real-world threat campaigns.

By aligning our threat model with these operational techniques, we ensure that the obfuscated code evaluated in our pipeline closely reflects the complexity and evasiveness of real-world adversaries. In turn, this allows us to rigorously assess whether LLMs can reliably identify vulnerabilities *under adversarial compilation conditions*, a crucial benchmark for their deployment in security-critical domains.

5. LLVM-based obfuscation and benchmarking

Code obfuscation plays a critical role in cybersecurity, serving both defensive and offensive purposes. While developers may use obfuscation to protect intellectual property and improve software resilience, adversaries often exploit the same techniques to evade AI-based analysis. With the growing adoption of LLMs in automated vulnerability detection, understanding how obfuscation interferes with their capabilities is an essential area of research.

This chapter presents an empirical study that evaluates the robustness of three LLMs: ChatGPT-4o, Claude Sonnet 4, and Gemini Flash 2.5 in analyzing C functions. Our benchmarking pipeline automates code transformation and obfuscation via a custom script named `obfuscate.sh`. This script also copies the intermediate files and artifacts into designated folders to ensure clean and organized output. The process is illustrated in Fig. 2.

Table 3
Obfuscation techniques that were used in our study.

Technique	Description	C code example

Bogus Control Flow (BCF)	Inserts opaque conditional branches that are never executed, increasing control flow complexity. Often used to mislead static analysis.	`int add(int a, int b) {`
		`  if (a == b && a != b) {`
		`  printf(‘‘Fake path’’);`
		`  }`
		`  return a + b;`
		`}`

Control Flow Flattening	Replaces structured control flow with a dispatcher loop and switch-case. Makes decompilation and CFG recovery difficult.	`int compute(int x) {`
		`  int state = 0;`
		`  while (1) {`
		`  switch(state) {`
		`  case 0: x += 1; state =`
		`1; break;`
		`  case 1: x *= 2; return x;`
		`  }`
		`  }`
		`}`

Instruction Substitution	Replaces simple instructions with semantically equivalent sequences to avoid detection. Alters opcode patterns.	`// Original:`
		`x = y + z;`
		`// Obfuscated:`
		`x = y;`
		`x += z;`

Basic Block Splitting	Splits a basic block into smaller ones with jumps in between to break recognizable patterns.	`int multiply(int a, int b) {`
		`  int result = a * b;`
		`  goto part2;`
		`part2:`
		`  return result;`
		`}`

5.1. Dataset and pre-processing

We use a balanced subset of the Devign dataset (Zhou et al., 2019), consisting of 40 functions in total: 20 labeled as vulnerable and 20 as safe. Before applying obfuscation, we manually modified each function to make it compilable using clang. This involved defining missing types, headers, and struct declarations. However, we ensured that the original logic of each function remained unchanged to preserve the ground truth vulnerability labels.

5.2. Obfuscation pipeline

We employ the open-source Obfuscator-LLVM (Obfuscator-LLVM Team, 2014), a C++-based tool compatible with LLVM-14. This toolchain is freely available, actively maintained, and easily reproducible across different environments and it provides four configurable obfuscation techniques that were shown in Table 3:

Focusing on these four transformations does not narrow the scope of our study. Similar techniques are well known in the research community and have also been observed in real-world malware, making them highly relevant for evaluating the robustness of LLMs against adversarial code transformations. Our selection was made to cover different layers of obfuscation: control flow (bogus control flow, flattening), semantics (instruction substitution), and syntax (basic block splitting). The generated obfuscated outputs and binary files were consistent across runs and formed the basis for our LLM analysis. As LLMs cannot directly process binary files, we further used the LLVM intermediate representation to enable experimentation. These transformations were applied to the selected Devign functions. Obfuscation was fully automated using a custom script, which compiles the original functions, applies the selected passes via ‘opt‘, and outputs transformed `.ll` and binary files. The process is illustrated in Fig. 3. All scripts, data, configuration files, resulting obfuscated code, and evaluation results are made publicly available in our GitHub repository.[1]

5.3. Evaluation setup

To evaluate the robustness of LLMs against compiler-level obfuscation, we tested three state-of-the-art models: ChatGPT-4o, Gemini Flash 2.5, and Claude Sonnet 4. Each function, both in its original and obfuscated form, was submitted to the models in a clean zero shot setting. Additionally, the corresponding .c source files were provided to the models prior to the LLVM IR representations in order to ensure that the models correctly understood the vulnerability status of the function before any compilation or transformation was applied. Due to the generative and non-deterministic nature of LLMs, we repeated each query three times for every function and used the majority prediction (‘secure’ or ‘insecure’) as the final classification. During experimentation, we observed that as chat sessions became longer, the models began to hallucinate due to memory leakage or context accumulation. To mitigate this effect and preserve the integrity of the zero-shot setting, each result was submitted in a separate, fresh chat session, and previous sessions were deleted. The prompt we used to query LLMs can be found in Appendix B. We recorded true positives (TP) and false negatives (FN) for both the original and obfuscated vulnerable samples.

5.4. Results and observations

The results reveal a notable decline in vulnerability detection performance across all three models following obfuscation. For the original

[1] Available at https://github.com/nikeboke/llm-llvm-adversarial (Böke and Torka, 2025).

Table 4
Performance of LLMs before and after obfuscation.

Model	Obfuscation	Accuracy	Precision	Recall (TPR)	Specificity (TNR)	F1-score
ChatGPT-4o	Before	0.800	0.772	0.850	0.750	0.810
	After	0.525	0.516	0.800	0.250	0.627
Gemini Flash 2.5	Before	0.675	0.652	0.750	0.600	0.698
	After	0.575	0.551	0.800	0.350	0.653
Claude Sonnet 4	Before	0.650	0.636	0.700	0.600	0.666
	After	0.600	0.560	0.900	0.300	0.692

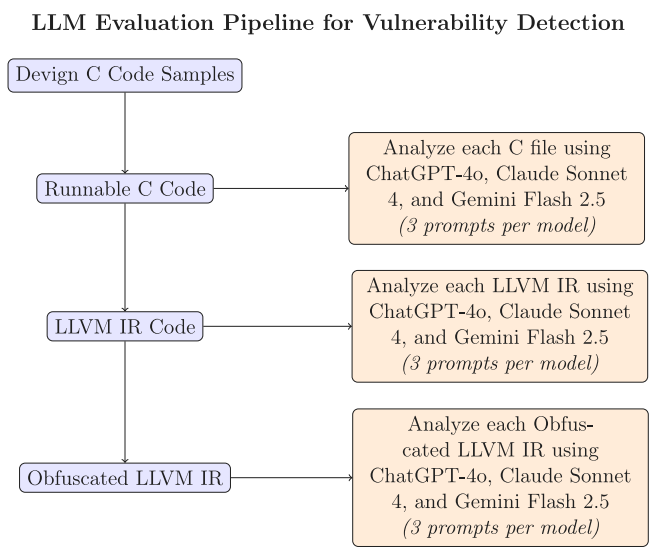

Fig. 3. LLM Evaluation Pipeline for Vulnerability Detection.
Evaluation pipeline for LLM-based vulnerability detection. The same samples are progressively transformed and analyzed using three different LLMs across multiple representations (C, LLVM IR, and obfuscated LLVM IR).

un-obfuscated functions, ChatGPT-4o achieved an accuracy of 0.800 with a precision of 0.772, recall of 0.850, specificity of 0.750, and F1-score of 0.810. Gemini Flash 2.5 followed with an accuracy of 0.675, and Claude Sonnet 4 at 0.650, both showing balanced recall and specificity. After obfuscation, ChatGPT-4o's accuracy dropped significantly to 0.525, and specificity fell to 0.250, indicating a much higher false positive rate. Gemini Flash 2.5 also showed reduced performance, with precision decreasing from 0.652 to 0.551 and specificity from 0.600 to 0.350. Claude Sonnet 4, while maintaining a strong recall of 0.900 after obfuscation, experienced a drop in specificity from 0.600 to 0.300. Further results can be found in Table 4. These findings underscore how compiler-level obfuscation impairs LLM-based vulnerability detection, particularly by reducing the models' ability to distinguish vulnerable code.

Overall, our findings highlight a significant vulnerability in the current generation of LLM-based malware and vulnerability detectors. These models are effective on clean, human-readable code, but their reliability degrades substantially under compiler level adversarial transformations. This gap underscores the need for training LLMs on compiler level representations and for integrating more robust adversarial defense mechanisms in AI-driven security pipelines.

5.5. Comparison across LLMs

Our comparative evaluation of ChatGPT-4o, Claude Sonnet 4, and Gemini Flash 2.5 reveals substantial differences in how each model responds to compiler-level obfuscation. While all three LLMs perform well on clean, unobfuscated functions, their robustness declines once LLVM-based transformations are applied.

Before obfuscation, ChatGPT-4o demonstrated the strongest overall performance with the highest accuracy of 0.800, precision of 0.772, recall of 0.850, specificity of 0.750, and F1-score of 0.810. Gemini Flash 2.5 followed with an accuracy of 0.675, showing balanced recall and precision, while Claude Sonnet 4 achieved slightly lower performance at 0.650 accuracy but with consistent scores across metrics.

After applying compiler-level obfuscation, performance dropped consistently across all models. ChatGPT-4o experienced a substantial decline in accuracy (0.800 to 0.525) and specificity (0.750 to 0.250), indicating a sharp increase in false positives. Claude Sonnet 4 achieved the highest recall post-obfuscation at 0.900, correctly identifying 18 out of 20 benign functions, but its specificity dropped to 0.300. Despite this trade-off, it ultimately achieved the highest F1-score after obfuscation (0.692). Gemini Flash 2.5 maintained a more balanced trade-off between recall (0.800) and specificity (0.350), offering a more moderate and consistent response to obfuscation.

These results underscore different model strengths: **ChatGPT-4o** was the most accurate and precise on clean code; **Claude Sonnet 4** prioritized vulnerability detection under obfuscation, achieving the highest recall and F1-score; and **Gemini Flash 2.5** balanced recall and specificity more evenly, making it a stable choice under adversarial conditions.

These results underscore that while LLMs retain some ability to detect obfuscated code patterns, their capacity to reliably identify vulnerabilities degrades significantly. Obfuscation induces a shift in model behavior toward falsely labeling vulnerable functions as benign, thereby increasing false negatives. This undermines the practical reliability of these models in real-world secure code bases, where undetected vulnerabilities pose serious risks.

In conclusion, future LLM-based security systems must incorporate adversarial robustness into their evaluation benchmarks. Training on obfuscated or intermediate representations, such as LLVM IR, and integrating obfuscation-aware pretraining may help counteract the adversarial effects of compiler-level transformations.

6. Limitations

This study is subject to several limitations that should be considered when interpreting the results. First, the analysis relies exclusively on the three LLMs ChatGPT-4o, Gemini Flash 2.5, and Claude Sonnet 4. The restriction to these LLMs inherently limits the generalizability of our findings, as other LLMs may behave differently when confronted with the same tasks.

Second, the generative nature of LLMs introduces variability in the answers they produce. Outputs can differ across runs even under identical input conditions, raising challenges for reproducibility and consistency. To mitigate this, we conducted three passes for each file; however, this repetition may not be sufficient to fully capture the variability of generative outputs. In addition, our analysis was based exclusively on zero-shot prompting, which, while providing an

unbiased baseline, may not reflect the full range of model capabilities under different prompting strategies.

Third, we focus only on `.ll`-files as part of the LLVM IR representation and not on the binary files (e.g., .exe). This choice was necessary because the evaluated LLMs are currently not able to directly process binary files and would first need to be fine-tuned for that purpose, which lies beyond the scope of this study. Furthermore, reverse engineering binary files into `.ll`-files is also not part of our experimental pipeline, as this step would exceed the scope of the work. While this ensures feasibility and consistency in the experimental setup, it also narrows the applicability of our results, as real-world usage often involves only the binary files.[2]

Fourth, our evaluation is limited to static analysis only. While hybrid or dynamic analyses could provide deeper insights into runtime behavior and resistance to obfuscation, they were deliberately excluded from the scope of this work. Current large-language models are not yet well equipped to process runtime artifacts such as syscall traces, memory states, or dynamic control-flow graphs in a reliable and generalizable manner, particularly in zero-shot evaluation settings. As such, we focused on static code inspection and outline hybrid analysis as a promising avenue for future research.

Fifth, we restricted the evaluation to the four obfuscation techniques that were implemented in our tool. Although this constraint ensures technical feasibility and consistency, it possibly reduces the breadth of the conclusions, as other obfuscation strategies were not considered.

Sixth, we only use 40 C source files from the Devign dataset, which limits the scope of our analysis. However, limiting ourselves to a subset cannot cover the entire diversity and complexity of software vulnerabilities, which limits the generalizability of our results and may introduce dataset-specific biases.

7. Conclusion & outlook

Our findings show that while LLMs such as ChatGPT-4o, Gemini Flash 2.5, and Claude Sonnet 4 can reliably detect vulnerabilities in clean C code, their effectiveness degrades significantly under compiler-level obfuscation. Basic syntactic and semantic obfuscation often fails to deceive these models, but transformations at the LLVM IR level consistently cause misclassifications in both vulnerable and safe functions. This shows that LLMs are still highly sensitive to changes in structural code, even when functionality is preserved.

These findings align with recent studies that reveal similar weaknesses. Prior work (Li et al., 2023) shows that even state-of-the-art models can yield incorrect answers in a significant number of cases due to small changes in code structure or naming. Further evidence (Steenhoek et al., 2024) demonstrates that LLMs struggle to reason about the code semantics essential for vulnerability identification, and that improvements in prompts, model size, or fine-tuning have not yet resolved these issues.

Given these limitations, we emphasize the need for LLMs that are specifically trained to understand and reason over low-level code representations, such as LLVM IR. Unlike general-purpose models, such specialized LLMs could be more resilient to compiler-level obfuscation and better equipped to detect embedded vulnerabilities. Developing such models represents a promising direction for future research toward more robust and trustworthy AI-driven security analysis.

Furthermore, compiler transformations can contribute to the adversarial hardening of LLM-based security tools. By systematically generating structurally diverse yet semantically equivalent code samples, researchers can create more comprehensive training and evaluation datasets. This approach can improve model generalization and reduce reliance on superficial code patterns. Recent work has shown that code-oriented language models are vulnerable to adversarially perturbed code generated via structural transformations, as demonstrated by Jha and Reddy (2022). Complementing these findings, Bielik and Vechev proposed adversarial training and representation refinement techniques to improve the robustness of code models against semantically-preserving attacks (Bielik and Vechev, 2020). Additionally, complementary methods such as self-evaluation without fine-tuning have proven effective in reducing the success rate of adversarial inputs against both open-source and closed-source LLMs (Brown et al., 2024). These directions underscore the broader potential of compiler technologies in advancing software protection, code attribution, and the development of secure and reliable AI systems.

In addition to improving detection capabilities, compiler-based obfuscation techniques can also be explored as defensive mechanisms. For instance, intentional obfuscation during compilation can help protect proprietary software from reverse engineering and unauthorized access (Collberg et al., 1997). Software watermarking represents another relevant application, where unique and resilient identifiers are embedded at the compiler level to trace software origin and detect unauthorized redistribution (Collberg and Nagra, 2009).

As part of our commitment to transparency and reproducibility, all artifacts used in our experiments, including code, scripts, prompt templates, and LLVM pass configurations, have been curated and will be made publicly available under an open license (Böke and Torka, 2025). These resources are intended to support replication and enable the research community to build upon our findings.

CRediT authorship contribution statement

Ekin Böke: Writing – original draft, Visualization, Validation, Methodology. **Simon Torka:** Writing – review & editing, Supervision.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT-4o, Gemini Flash 2.5 and Claude Sonnet 4 to assist with code analysis and grammar refinement. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Code obfuscation examples

See Tables A.5–A.7.

Appendix B. LLM prompt templates

See Table B.1.

Data availability

We have made data available in our public Github repo.

[2] Toor (2022) provides a solution to obtain `.ll`-files from binary files.

Table A.5
Examples of syntactic obfuscation techniques in C.

Technique	Description	C code example
Variable Renaming	Changes variable and function names to meaningless identifiers to mislead static analysis tools.	`int t9s_y1(int x1, int x2) { return x1 + x2; }`
Dead Code Insertion	Adds unnecessary code that does not affect functionality but confuses analysis tools.	`int calculate(int a, int b) { int x = 10; // unused variable int y = x * 5; // redundant calculations if (y >50) printf(''Good for nothing.''); return a + b; }`
Formatting Obfuscation	Alters code formatting (spacing, indentation) while keeping logic unchanged.	`int calculate(int a,int b){return a+b;}`

Table A.6
Examples of semantic obfuscation techniques in C.

Technique	Description	C code example
Control Flow Distortion	Alters the program's execution flow, making it harder to analyze and understand the logic.	`int calculate(int a, int b) { int state = 0, result; while (1) { switch (state) { case 0: state = 1; break; case 1: result = a + b; state = 2; break; case 2: return result; } } }`
Opaque Predicate	Introduces conditional statements that always evaluate to the same value to confuse analysis tools.	`int calculate(int a, int b) { int x = 1337; if ((x * x) % 1337 == 0) // always true return a + b; else return a - b; }`
Instruction Substitution	Replaces simple operations with more complex equivalents to obscure intent.	`int calculate(int a, int b) { int result = 0; for (int i = 0; i <abs(b); i++) result += a; if (b <0) result = -result; return result; }`

Table A.7
Examples of behavioral obfuscation techniques in C.

Technique	Description	C code example
Packing/Encryption	Encrypts or compresses code, making it unreadable until runtime, when it is dynamically unpacked.	`void encrypted_payload() { char payload[] = ''ajdjfjfjfj''; int key = 847584; for (int i = 0; i <strlen(payload); i++) payload[i] ^= key; printf(''Decrypted: %s'', payload); }`

(*continued on next page*)

Table A.7 (*continued*).

Technique	Description	C code example
Polymorphic Behavior	Code dynamically mutates its structure while maintaining the same functionality, making signature-based detection difficult.	`void poly_function(int a, int b) { int (*func_ptr)(int, int); if ((rand() % 2) == 0) func_ptr = &calculate; else func_ptr = &alternate_calculate; int result = func_ptr(a, b); printf(``Result: %d'', result); }`
Metamorphic Code	Rewrites itself during execution, ensuring that no two versions of the code are identical, even though behavior remains unchanged.	`void metamorphic_function(){ char *variants[] = {``Char1'', ``Char2''}; int choice = rand() % 2; printf(``Executing: %s'', variants[choice]); }`

Table B.1
Prompts used for each code type evaluated by the LLMs.

Prompt type	Prompt text
C-Code	Analyze the provided C source code for potential security vulnerabilities or signs of malicious behavior. Using exactly one of the following formats: • If the file is secure: Yes, the code is secure. • If the file is insecure: No, the code is insecure because [reason]. Answer briefly. Code:
LLVM-orig-code	Analyze the provided LLVM source code for potential security vulnerabilities or signs of malicious behavior. Using exactly one of the following formats: • If the file is secure: Yes, the code is secure. • If the file is insecure: No, the code is insecure because [reason]. Answer briefly. Code:
LLVM-obf-code	Analyze the provided obfuscated LLVM source code for potential security vulnerabilities or signs of malicious behavior. Using exactly one of the following formats: • If the file is secure: Yes, the code is secure. • If the file is insecure: No, the code is insecure because [reason]. Answer briefly. Code:

References

Akinsowon, T., Jiang, H., 2024. Leveraging large language models for behavior-based malware detection using deep learning. IHS Online Technical Papers. URL https://ihsonline.org/Portals/0/Tech%20Papers/2024_Papers/Akinsowon-Jiang_LeveragingLargeLanguageModels.pdf?ver=2RlcpRiD4bUoBIcb9SnOsg%3D%3D. (Accessed 01 February 2025).

Allix, K., Bissyandé, T.F., Klein, J., Traon, Y.L., 2016. Androzoo: Collecting millions of android apps for the research community. In: Proceedings of the 13th International Conference on Mining Software Repositories. MSR '16, ACM, New York, NY, USA, pp. 468–471. http://dx.doi.org/10.1145/2901739.2903508.

Anderson, M., Cafarella, M., 2016. Input selection for fast feature engineering. pp. 577–588. http://dx.doi.org/10.1109/ICDE.2016.7498272.

Anthropic, 2024. Introducig claude 4. https://www.anthropic.com/news/claude-4. (Accessed July 2025).

Bielik, P., Vechev, M., 2020. Adversarial robustness for code. arXiv preprint arXiv:2002.04694.

Böke, E., Torka, S., 2025. LLM-LLVM-Adversarial. https://github.com/nikeboke/llm-llvm-adversarial. (Accessed 13 September 2025).

Brezinski, K., 2023. Metamorphic malware and obfuscation: A survey of obfuscation techniques used by mutation engines. ISRN Comput. Sci. 2023, 1–19. http://dx.doi.org/10.1155/2023/8227751, URL https://onlinelibrary.wiley.com/doi/10.1155/2023/8227751.

Brown, H., Lin, L., Kawaguchi, K., Shieh, M., 2024. Self-evaluation as a defense against adversarial attacks on LLMs. arXiv preprint arXiv:2407.03234.URL https://arxiv.org/abs/2407.03234.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, N., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners. Adv. Neural Inf. Process. Syst. 33.

Collberg, C., Nagra, J., 2009. Surreptitious software: obfuscation, watermarking, and tamperproofing for software protection. Addison-Wesley.

Collberg, C., Thomborson, C., Low, D., 1997. A taxonomy of obfuscating transformations. Tech. Rep., (148), University of Auckland.

DeepMind, G., 2024. Gemini 2.5. https://deepmind.google/technologies/gemini/. (Accessed July 2025).

Demirci, D., şahin, N., şirlancis, M., Acartürk, C., 2022. Static malware detection using stacked BiLSTM and GPT-2. IEEE Access 10, 58488–58502. http://dx.doi.org/10.1109/ACCESS.2022.3179384.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. http://dx.doi.org/10.48550/arXiv.1810.04805, arXiv preprint arXiv:1810.04805.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Liu, T., Jiang, D., Zhou, M., 2020. Codebert: A pre-trained model for programming and natural languages. pp. 1536–1547. http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139.

Hex-Rays, 2023. IDA pro disassembler. https://www.hex-rays.com/products/ida/.

Jha, A., Reddy, C.K., 2022. CodeAttack: Code-based adversarial attacks for pre-trained programming language models. arXiv preprint arXiv:2206.00052.

Junod, P., Rinaldini, J., Wehrli, J., Michielin, J., 2015. Obfuscator-LLVM: Software protection for the masses. In: Proceedings of the IEEE/ACM 1st International Workshop on Software Protection. SPRO'15, IEEE, pp. 3–9.

Lattner, C., Adve, V., 2004. LLVM: A compilation framework for lifelong program analysis and transformation. In: Proceedings of the 2004 International Symposium on Code Generation and Optimization. CGO'04, IEEE Computer Society, pp.

75–86. http://dx.doi.org/10.1109/CGO.2004.1281665, URL https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf.

Li, Y., Li, C., Li, Y., Chen, K., Zhang, Y., Wang, Y., Zhu, S., Wang, X., 2023. SecLLMHolmes: Probing LLMs for security vulnerability detection and reasoning. arXiv preprint arXiv:2312.12575. URL https://arxiv.org/abs/2312.12575.

Li, W., Zhang, J., Huang, C., Wang, M., 2022. Multi-view representation learning from malware to defend against adversarial variants. arXiv preprint arXiv:2210.15429. URL https://arxiv.org/abs/2210.15429.

Li, C., Zhang, J., Wang, L., Xu, X., 2024. Automated vulnerability detection using deep learning techniques. arXiv preprint arXiv:2410.21968.

LLVM Project, 2025. LLVM language reference manual. URL https://llvm.org/docs/LangRef.html.

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al., 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. arXiv preprint arXiv:2102.04664.

Lyu, C., Yan, L., Xing, R., Li, W., Samih, Y., Ji, T., Wang, L., 2024. Large language models as code executors: An exploratory study. arXiv preprint arXiv:2410.06667. URL https://arxiv.org/abs/2410.06667.

National Security Agency, 2019. Ghidra Software Reverse Engineering Framework. https://ghidra-sre.org/.

Obfuscator-LLVM Team, 2014. Obfuscator-LLVM. https://github.com/eshard/obfuscator-llvm.

OpenAI, 2024. ChatGPT-4o model by openai. https://chat.openai.com. (Accessed March 2025).

Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., 2017. Malware detection by eating a whole EXE. arXiv preprint arXiv:1710.09435.

Rahali, A., Akhloufi, M.A., 2021. Malbert: Using transformers for cybersecurity and malicious software detection. arXiv preprint arXiv:2103.03806. URL https://arxiv.org/abs/2103.03806.

Revay, G., 2023. Don't flatten yourself: restoring malware with control-flow flattening obfuscation. In: Virus Bulletin Conference (VB2023). Virus Bulletin, London, United Kingdom, URL https://www.virusbulletin.com/uploads/pdf/conference/vb2023/papers/Dont-flatten-yourself-restoring-malware-with-Control-Flow-Flattening-obfuscation.pdf.

Sharif, M., Lanzi, A., Giffin, J., Lee, W., 2008. Impeding malware analysis using conditional code obfuscation. In: Network and Distributed System Security Symposium. NDSS, URL https://llvm.org/pubs/2008-02-ImpedingMalwareAnalysis.pdf.

Singh, R., Gupta, P., Sharma, A., 2024. Malicious code detection using LLM. IEEE Trans. Dependable Secur. Comput. 21 (1), 85–96. http://dx.doi.org/10.1109/TDSC.2024.10670668.

Steenhoek, W., Siwakoti, Y., Yuan, M., Marinov, D., 2024. To err is machine: Vulnerability detection challenges LLM reasoning. arXiv preprint arXiv:2403.17218. URL https://arxiv.org/abs/2403.17218.

Toor, T.S., 2022. Decompilation of Binaries into LLVM IR for Automated Analysis. University of Waterloo, URL https://uwspace.uwaterloo.ca/bitstream/handle/10012/17976/Toor_Tejvinder.pdf.

VMware Threat Analysis Unit, 2019. Defeating compiler-level obfuscations used in APT10 malware. https://blogs.vmware.com/security/2019/02/defeating-compiler-level-obfuscations-used-in-apt10-malware.html. (Accessed 03 September 2025).

Zhang, P., Wu, C., Peng, M., Zeng, K., Yu, D., Lai, Y., Kang, Y., Wang, W., Wang, Z., 2023. Khaos: The impact of inter-procedural code obfuscation on binary diffing techniques. http://dx.doi.org/10.48550/arXiv.2301.11586, arXiv preprint arXiv:2301.11586. URL https://arxiv.org/abs/2301.11586. Submitted 27 Jan 2023.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, URL https://arxiv.org/abs/1909.03496.